

Term-2 1446 H

IT 462-Big Data Systems Final Report

Student Performance Dataset
Section 56325
Group #6

Rawan Alsaif	443200449
Ghala Al Askar	443200657
Reem Al Hawass	443200461
Omaimah Attowaim	443200766
Jana Al Suhaibani	443200428

Table of Contents

1	<i>Introduction</i>	<i>4</i>
2	<i>Data Selection.....</i>	<i>4</i>
3	<i>Data Preprocessing</i>	<i>6</i>
3.1	Data Cleaning	6
3.2	Data Integration (Not Applied)	7
3.3	Data Reduction (Not Applied).....	7
3.4	Data Transformation.....	7
4	<i>Reading the Student Performance CSV Dataset.....</i>	<i>8</i>
5	<i>RDD Operations</i>	<i>8</i>
5.1	RDD Transformations.....	8
5.2	RDD Actions.....	9
5.3	Findings.....	10
6	<i>SQL Operations</i>	<i>11</i>
6.1	SQL Queries	11
6.2	Findings.....	13
7	<i>Machine Learning Operations.....</i>	<i>14</i>
7.1	Reading and Preparing the Data	14
7.2	Random Forest	17
7.3	Decision Tree	18
7.4	Results.....	18
8	<i>Conclusion.....</i>	<i>19</i>
9	<i>References</i>	<i>20</i>

1 Introduction

In this project, we utilize a dataset sourced from Kaggle that focuses on student performance, providing insights into various factors influencing academic success [1]. We begin by preprocessing the data and preparing it for analysis. Next, we explore RDD operations which include both transformations and actions, accompanied by their findings. Following this, execute a variety of SQL queries, allowing us to extract meaningful insights from the dataset and present the associated findings. Finally, we implement different machine learning algorithms to predict students grade class. Our goal is to gain more insight into students' performance and how different factors affect it.

2 Data Selection

Students Performance Dataset is a structured dataset in Excel CSV format, which consists of 2,392 rows of students' data and 15 columns. It contains comprehensive information on high school students, detailing their demographics, study habits, parental involvement, extracurricular activities, and academic performance which will be shown in Table 1 below with its description. The target variable, GradeClass, classifies students' grades into distinct categories. The dataset is sourced from Kaggle [1].

Table 1 Dataset Description

Attribute	Type	Description
StudentID	Nominal	A unique identifier assigned to each student (1001 to 3392).
Age	Numerical	The age of the students ranges from 15 to 18 years.
Gender	Binary Nominal	Gender of the students, where 0 represents Male and 1 represents Female.
Ethnicity	Categorical	The ethnicity of the students, coded as follows: 0: Caucasian 1: African American 2: Asian 3: Other
ParentalEducation	Ordinal	The education level of the parents, coded as follows: 0: None 1: High School 2: Some College 3: Bachelor's 4: Higher
StudyTimeWeekly	Numerical	Weekly study time in hours, ranging from 0 to 20.

Absences	Numerical	Number of absences during the school year, ranging from 0 to 30.
Tutoring	Binary Nominal	Tutoring status, where 0 indicates No and 1 indicates Yes.
ParentalSupport	Ordinal	The level of parental support, coded as follows: 0: None 1: Low 2: Moderate 3: High 4: Very High
Extracurricular	Binary Nominal	Participation in extracurricular activities, where 0 indicates No and 1 indicates Yes.
Sports	Binary Nominal	Participation in sports, where 0 indicates No and 1 indicates Yes.
Music	Binary Nominal	Participation in music, where 0 indicates No and 1 indicates Yes.
Volunteering	Binary Nominal	Participation in volunteering, where 0 indicates No and 1 indicates Yes.
GPA	Numerical	Grade Point Average on a scale from 2.0 to 4.0, influenced by study habits, parental involvement, and extracurricular activities.
GradeClass	Ordinal	Classification of students' grades based on GPA: 0: 'A' (GPA ≥ 3.5) 1: 'B' ($3.0 \leq \text{GPA} < 3.5$) 2: 'C' ($2.5 \leq \text{GPA} < 3.0$) 3: 'D' ($2.0 \leq \text{GPA} < 2.5$) 4: 'F' (GPA < 2.0)

3 Data Preprocessing

Data preprocessing is a vital step in the data analysis pipeline, as it prepares the raw data for modeling and analysis. It involves several key activities, including data cleaning and transformation. Through effective data preprocessing, we can enhance the dataset's quality, enabling us to build more robust predictive models that can provide valuable insights into student performance. It sets the foundation for subsequent analysis and ensures that we approach our tasks with a clean and well-structured dataset.

3.1 Data Cleaning

In this section, we will focus on cleaning the dataset to ensure its accuracy and reliability. This involves identifying and addressing missing value and duplicates in the data.

- Check for Missing Values

As shown in Figure 1, there are no missing values in the dataset.



```
# Check for missing values
data.isnull().sum()
```

	0
StudentID	0
Age	0
Gender	0
Ethnicity	0
ParentalEducation	0
StudyTimeWeekly	0
Absences	0
Tutoring	0
ParentalSupport	0
Extracurricular	0
Sports	0
Music	0
Volunteering	0
GPA	0
GradeClass	0

dtype: int64

Figure 1 Check for Missing Values

- Check for Duplicate Rows

As shown in Figure 2, there are no duplicate rows in the dataset.



```
# Check for duplicate rows
duplicates = data.duplicated().sum()
print(f"Number of duplicate rows: {duplicates}")
```

Number of duplicate rows: 0

Figure 2 Check for Duplicate Rows

3.2 Data Integration (Not Applied)

Data integration is not necessary at this stage, as we are working with a single dataset (Students Performance Dataset) sourced entirely from Kaggle. This dataset already contains all the necessary information for analysis, ensuring that no additional data merging is required.

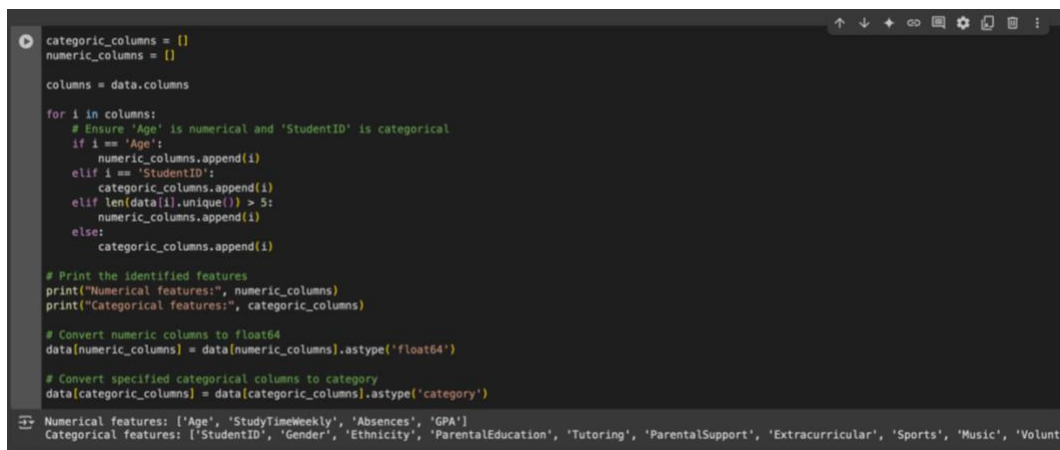
3.3 Data Reduction (Not Applied)

We do not need data reduction in this instance, as our dataset is suitable and contains all necessary information for analysis. The data is already well-structured, and there are no excessive features or redundant entries that require elimination.

3.4 Data Transformation

In this section, we will categorize the features of our dataset into numerical and categorical types, which is essential for subsequent analysis and modeling.

As shown in Figure 3 and Figure 4, the dataset includes four numerical features: Age, StudyTimeWeekly, Absences, and GPA. Additionally, it contains eleven categorical features: StudentID, Gender, Ethnicity, ParentalEducation, Tutoring, ParentalSupport, Extracurricular, Sports, Music, Volunteering, and GradeClass. This distinction highlights the different types of data represented in the dataset, which is crucial for subsequent analysis and modeling.



```

categoric_columns = []
numeric_columns = []

columns = data.columns

for i in columns:
    # Ensure 'Age' is numerical and 'StudentID' is categorical
    if i == 'Age':
        numeric_columns.append(i)
    elif i == 'StudentID':
        categoric_columns.append(i)
    elif len(data[i].unique()) > 5:
        numeric_columns.append(i)
    else:
        categoric_columns.append(i)

# Print the identified features
print("Numerical features:", numeric_columns)
print("Categorical features:", categoric_columns)

# Convert numeric columns to float64
data[numeric_columns] = data[numeric_columns].astype('float64')

# Convert specified categorical columns to category
data[categoric_columns] = data[categoric_columns].astype('category')

Numerical features: ['Age', 'StudyTimeWeekly', 'Absences', 'GPA']
Categorical features: ['StudentID', 'Gender', 'Ethnicity', 'ParentalEducation', 'Tutoring', 'ParentalSupport', 'Extracurricular', 'Sports', 'Music', 'Volunteering', 'GradeClass']

```

Figure 3 Convert Data Types



```

[ ] # Print the data types of each column
print(data.dtypes)

StudentID      category
Age            float64
Gender         category
Ethnicity      category
ParentalEducation  category
StudyTimeWeekly  float64
Absences       float64
Tutoring       category
ParentalSupport category
Extracurricular category
Sports         category
Music          category
Volunteering   category
GPA            float64
GradeClass     category
dtype: object

```

Figure 4 Data Types for Each Column

4 Reading the Student Performance CSV Dataset

Figure 5 below captures the process of loading a CSV file containing student performance data into a Spark DataFrame. The `dataset.show()` command displays a preview of the top 20 rows, providing insights into the structure and content of the data. Key fields include StudentID, Gender, Parental Education, Study Time Weekly, Absences, GPA, and Grade Class.

```
scala> val dataset = spark.read.option("header", "true").csv("C:\\spark\\Student_performance_data_1.csv")
dataset: org.apache.spark.sql.DataFrame = [StudentID: string, Age: string ... 13 more fields]

scala> dataset.show()
```

StudentID	Age	Gender	Ethnicity	ParentalEducation	StudyTimeWeekly	Absences	Tutoring	ParentalSupport	Extracurricular	Sports	Music	Volunteering	GPA	GradeClass
1001	17	1	0		219.833722807854713	7	1	2	0	0	1	0	2.929198591667681	2.0
1002	18	0	0		115.08875605586761	0	0	1	0	0	0	0	3.0429148310436377	1.0
1003	15	0	2		314.21086976881226	26	0	2	0	0	0	0	0.1126022504661915	4.0
1004	17	1	0		310.028829473988215	14	0	3	1	0	0	0	0.2.0842181397029484	3.0
1005	17	1	0		214.6724952729713305	17	1	3	0	0	0	0	0.1.2888611817953875	4.0
1006	18	0	0		118.191213505250186	0	0	1	1	0	0	0	0.1.08018361048639377	1.0
1007	15	0	1		1115.461680478609295	18	0	3	0	1	0	0	0.2.7482374148915831	2.0
1008	15	1	1		415.420496305888974	22	1	1	1	0	0	0	0.1.366142712316461	4.0
1009	17	0	0		014.562087558047763	1	0	2	0	1	0	0	1.2.086619109513569	2.0
1010	16	1	0		1118.40046363897282	0	0	3	1	0	0	0	0.1.5734742103297666	0.0
1011	17	0	0		111.851363655296536	11	0	1	0	0	0	0	0.2.1473716258185144	3.0
1012	17	0	0		117.59848581924829	15	0	2	0	0	0	0	1.1.5595945190402773	4.0
1013	17	0	1		110.038711615617213	21	0	3	1	0	0	0	0.1.528877814874888	4.0
1014	17	0	1		212.101425668754876	21	0	4	0	1	0	0	0.1.7015809583348785	4.0
1015	18	1	0		111.197810636915708	9	1	2	0	0	0	0	0.2.396788117124796	3.0
1016	15	0	0		219.728109710723563	17	1	0	0	1	0	0	0.1.3415207165346672	4.0
1017	18	0	1		110.09865681788902	14	0	2	1	1	0	0	0.2.2321782777159762	3.0
1018	18	1	0		013.520232080557235	16	1	2	0	0	0	0	0.1.3894041765900335	4.0
1019	18	0	1		316.28465886609359	29	0	2	1	0	0	0	1.0.4695833237987804	4.0
1020	17	0	0		110.83520639820308	9	0	2	0	0	1	0	0.2.3957048945306996	3.0

only showing top 20 rows

Figure 5 Reading the Dataset

5 RDD Operations

5.1 RDD Transformations

- Transformation 1: Sort by GPA

This transformation will reorder the students in the dataset based on their GPAs in ascending order, as shown in Figure 6.

```
scala> val sortedGPA = dataset.orderBy($"GPA")
sortedGPA: org.apache.spark.sql.Dataset[org.apache.spark.sql.Row] = [StudentID: string, Age: string ... 13 more fields]
```

Figure 6 RDD Transformation 1

- Transformation 2: Group by Grade Class

This transformation will give us the number of students in each grade class, as shown in Figure 7.

```
scala> val grades = dataset.groupBy($"GradeClass").count()
grades: org.apache.spark.sql.DataFrame = [GradeClass: string, count: bigint]
```

Figure 7 RDD Transformation 2

- Transformation 3: Filter Students Who Passed

This transformation will filter students with a GPA greater than 2.0, as shown in Figure 8.

```
scala> val PassedStudent = dataset.filter($"GPA" >= 2.0)
PassedStudent: org.apache.spark.sql.Dataset[org.apache.spark.sql.Row] = [StudentID: string, Age: string ... 13 more fields]
```

Figure 8 RDD Transformation 3

- Transformation 4: Categorizes students based on their GPA into three groups: "High" ($\text{GPA} \geq 3.0$), "Medium" ($2.0 \leq \text{GPA} < 3.0$), and "Low" ($\text{GPA} < 2.0$). It outputs a tuple for each student with the category and a count of 1, as shown in Figure 9.

```
scala> val categorizedGPA = dataset.rdd.map { row =>
  |   val gpa = row.getAs("GPA").toString.toDouble
  |   val category = if (gpa >= 3.0) "High" else if (gpa >= 2.0) "Medium" else "Low"
  |   (category, 1)
  | }
categorizedGPA: org.apache.spark.rdd.RDD[(String, Int)] = MapPartitionsRDD[21] at map at <console>:23
```

Figure 9 RDD Transformation 4

- Transformation 5: Prepare Absences for Maximum Calculation
This transformation will convert the "Absences" column to an integer type using withColumn and cast("int") which will enable accurate identification of the highest number of absences [2], as shown in Figure 10

```
scala> val datasetWithAbsencesAsInt = dataset.withColumn("Absences", col("Absences").cast("int"))
datasetWithAbsencesAsInt: org.apache.spark.sql.DataFrame = [StudentID: string, Age: string ... 13 more fields]
```

Figure 10 RDD Transformation 5

- Transformation 6: casts the "GPA" column to a Double type, ensuring that any calculations involving GPA are performed correctly, as shown in Figure 11.

```
scala> val gpaPrepared = absencesPrepared.withColumn("GPA", col("GPA").cast("double"))
gpaPrepared: org.apache.spark.sql.DataFrame = [StudentID: string, Age: string ... 13 more fields]
```

Figure 11 RDD Transformation 6

- Transformation 7: filters the DataFrame to create an RDD containing only the rows where students have zero absences (Perfect Attendance), as shown in Figure 12. This allows for focused analysis on students with perfect attendance. Similarly, this transformation filters the DataFrame to create an RDD of students who have one or more absences (Non-Perfect Attendance). This provides a comparative group for analysis.

```
scala> val perfectAttendanceFilter = gpaPrepared.rdd.filter(row => row.getAs[Int]("Absences") == 0)
perfectAttendanceFilter: org.apache.spark.rdd.RDD[org.apache.spark.sql.Row] = MapPartitionsRDD[51] at filter at <console>:23

scala> val nonPerfectAttendanceFilter = gpaPrepared.rdd.filter(row => row.getAs[Int]("Absences") > 0)
nonPerfectAttendanceFilter: org.apache.spark.rdd.RDD[org.apache.spark.sql.Row] = MapPartitionsRDD[52] at filter at <console>:23
```

Figure 12 RDD Transformation 7

5.2 RDD Actions

- Action 1: Get Minimum GPA
This action will retrieve the student with the lowest GPA, as shown in Figure 13.

```
scala> sortedGPA.first()
res15: org.apache.spark.sql.Row = [1088,15,0,2,1,0.008030880506797455,27,0,1,0,0,1,0,0,0,4,0]
```

Figure 13 RDD Action 1

- Action 2: Display Student Count by Grade Class
This action displays the count of students in each grade class, as shown in Figure 14.

```
scala> grades.take(5)
res18: Array[org.apache.spark.sql.Row] = Array([1,0,260], [0,0,107], [4,0,1211], [2,0,391], [3,0,414])
```

Figure 14 RDD Action 2

- Action 3: Count Passed Students

This action will count the number of students with GPAs greater than 2.0, as shown in Figure 15.

```
scala> PassedStudent.count()
res0: Long = 1118
```

Figure 15 RDD Action 3

- Action 4: Show Average Absence

This action will display the average absence per grade class, as shown in Figure 16.

```
scala> averageAbsencesByGradeClass.show()
+-----+-----+
|GradeClass|AverageAbsences|
+-----+-----+
|1.0|5.312267657992565|
|0.0|5.747663551401869|
|4.0|20.786952931461602|
|2.0|7.250639386189258|
|3.0|11.427536231884059|
+-----+-----+
```

Figure 16 RDD Action 4

- Action 4: Counts the number of students in each GPA category ("High", "Medium", "Low"), as shown in Figure 17.

```
scala> val gpaCategoryCounts = categorizedGPA.reduceByKey(_ + _).collect()
gpaCategoryCounts: Array[(String, Int)] = Array((High,321), (Medium,797), (Low,1274))

scala> gpaCategoryCounts.foreach(println)
(High,321)
(Medium,797)
(Low,1274)
```

Figure 17 RDD Action 5

5.3 Findings

1. By sorting the dataset in ascending order by GPA, we discovered that the minimum GPA was 0.0.
2. In terms of student distribution across grade classes, A, which is represented by 0, had 107 students, B, represented by 1, had 269 students, C, represented by 2, had 391 students, D, represented by 3, had 414 students, and F, represented by 4, had 1211 students.
3. Filtering the dataset for students with a GPA of 2.0 or higher, which indicates that they have passed, revealed that 1118 students were successful.
4. By calculating the average number of absences for each grade class, we found that grade class F, represented by four, had the highest average absences at 20.78, while grade class B, represented by one, had the lowest average absences at 5.31. This suggests that higher absenteeism may negatively impact students' grades, as increased absences are associated with lower performance in grade class.
5. By categorizing students based on GPA, we found that 321 students fall into the "High" GPA category, 797 in the "Medium" category, and 1274 in the "Low" category. This indicates that a significant majority of students (approximately 62.5%) have a low GPA, suggesting potential challenges in academic performance and the need for targeted support to improve student outcomes.

Overall, the findings from both the transformations and actions provide a thorough understanding of student performance, attendance, and class dynamics. This analysis can inform educational strategies, support systems, and resource allocations aimed at enhancing student outcomes and academic success. By addressing the insights gained from this dataset, educational institutions can make informed decisions to better serve their student populations.

6 SQL Operations

6.1 SQL Queries

- SQL 1: Average weekly study time per grade class:
It will help us identify how the average study time differs based on the students' grade class, as shown in Figure 18.

```
scala> val GradeandStudyTime = spark.sql("SELECT GradeClass, AVG(StudyTimeWeekly) FROM students GROUP BY GradeClass ORDER BY GradeClass")
GradeandStudyTime: org.apache.spark.sql.DataFrame = [GradeClass: double, avg(StudyTimeWeekly): double]

scala> GradeandStudyTime.show()
+-----+-----+
|GradeClass|avg(StudyTimeWeekly)|
+-----+-----+
|0.0|11.854925932154103|
|1.0|11.122335245132081|
|2.0|10.10640387913262|
|3.0|9.757962645413912|
|4.0|9.184821605001872|
+-----+-----+
```

Figure 18 SQL Query 1

- SQL 2: Average, Maximum and Minimum GPA for Students with high absence (more than 10) and low absence (less than or equal 10):
It will help identify how the absence affects the GPA, as shown in Figure 19.

```
scala> val lowabsence = spark.sql("SELECT AVG(GPA), MAX(GPA), MIN(GPA) FROM students WHERE Absences <= 10")
lowabsence: org.apache.spark.sql.DataFrame = [avg(GPA): double, max(GPA): double ... 1 more field]

scala> lowabsence.show()
+-----+-----+-----+
|avg(GPA)|max(GPA)|min(GPA)|
+-----+-----+-----+
|2.8568517647798415|4.0|1.6338115616978532|
+-----+-----+-----+

scala> val highabsence = spark.sql("SELECT AVG(GPA), MAX(GPA), MIN(GPA) FROM students WHERE Absences > 10")
highabsence: org.apache.spark.sql.DataFrame = [avg(GPA): double, max(GPA): double ... 1 more field]

scala> highabsence.show()
+-----+-----+-----+
|avg(GPA)|max(GPA)|min(GPA)|
+-----+-----+-----+
|1.3869152520251558|3.138784394067383|0.0|
+-----+-----+-----+
```

Figure 19 SQL Query 2

- SQL 3: Average grade class for students based on parental support level
It will help identify how parental support and the grade class related, as shown in Figure 20.

```
scala> val avgGPAbyParentSupport = spark.sql("SELECT ParentalSupport, AVG(GradeClass) FROM students GROUP BY ParentalSupport ORDER BY ParentalSupport")
avgGPAbyParentSupport: org.apache.spark.sql.DataFrame = [ParentalSupport: int, avg(GradeClass): double]

scala> avgGPAbyParentSupport.show()
+-----+-----+
|ParentalSupport| avg(GradeClass)|
+-----+-----+
|0| 3.292452838188679|
|1| 3.149284253578732|
|2| 3.0178675675675677|
|3| 3.8378766148682583|
|4| 2.7886614172228347|
+-----+-----+
```

Figure 20 SQL Query 3

- SQL 4: Average GPA for Students who are involved in extracurricular activities (music, volunteering, sports, or extracurricular) and students who aren't
It will help identify how the extracurricular activities affect the GPA, as shown in Figure 21.

```
scala> val avgGPA = spark.sql("SELECT AVG(GPA) FROM students WHERE Music = 1 OR Volunteering = 1 OR Sports = 1 OR Extracurricular = 1")
avgGPA: org.apache.spark.sql.DataFrame = [avg(GPA): double]

scala> avgGPA.show()
+-----+
| avg(GPA) |
+-----+
|1.9671532611660856|
+-----+

scala> val avgGPA = spark.sql("SELECT AVG(GPA) FROM students WHERE Music = 0 AND Volunteering = 0 AND Sports = 0 AND Extracurricular = 0")
avgGPA: org.apache.spark.sql.DataFrame = [avg(GPA): double]

scala> avgGPA.show()
+-----+
| avg(GPA) |
+-----+
|1.752377171935756|
+-----+
```

Figure 21 SQL Query 4

- SQL 5: Average Absences, GPA, and Study Time by Ethnicity
It retrieves the average number of absences, average GPA, and average weekly study time for students categorized by their ethnicity, as shown in Figure 22.

```
scala> val avgGPANoExtracurricular = spark.sql("SELECT Ethnicity, AVG(Absences), AVG(GPA), AVG(StudyTimeWeekly) FROM students GROUP BY Ethnicity ")
avgGPANoExtracurricular: org.apache.spark.sql.DataFrame = [Ethnicity: int, avg(Absences): double ... 2 more fields]

scala> avgGPANoExtracurricular.show()
+-----+-----+-----+-----+
|Ethnicity| avg(Absences)| avg(GPA)| avg(StudyTimeWeekly)|
+-----+-----+-----+-----+
|1|14.158215810141987| 1.946437163919105| 9.929419669895118|
|3|14.166666666666666| 1.9476957238837224| 10.31573714768453|
|2|14.438297872140426| 1.9221251422884317| 9.438488332628227|
|0|14.886959403479702| 1.8759845956425315| 9.748488655265844|
+-----+-----+-----+-----+
```

Figure 22 SQL Query 5

6.2 Findings

1. Greater study time is positively related to higher grades and better academic performance, where students who have high grades spend more time studying than other students with lower grades. Grade classes A and B have the highest average weekly study times where both are above 11 hours, while grade F has the lowest average which is (approximately 9.18) hours.
2. There is an inverse relationship between absence and student performance. Low absence leads to higher grades, while increased absence leads to lower academic performance. Students with low absence (less than 10) have a high average GPA (approximately 2.86), maximum GPA (4.0), and minimum GPA (approximately 1.63). In contrast, students with high absence (more than 10) have a significantly lower average GPA (approximately 1.39), maximum GPA (3.14), and minimum GPA (0.0).
3. Students with no or low parental support are getting grades between D and F, and students with moderate to very high parental support get grades between C and D. This highlights that there is a positive relationship between parental support and student grades.
4. Students who participate in activities such as music, volunteering, sports, or extracurricular activities have higher average GPA (approximately 1.97) compared to students who don't participate in these activities, where their average GPA is (approximately 1.75). This shows that participating in these activities can lead to better grades.
5. The analysis shows that there is a very slight difference among ethnic groups in their average GPA, absence, and weekly study time. African American student and other ethnic students have the highest average GPA (1.95), while ethnicity Caucasian students have the lowest average GPA (1.88). The average weekly study times across the groups range from 9.41 to 10.32 hours, where students from other ethnicities have the highest average and Asian students have the lowest. The student groups have an average of approximately 14 to 15 absences, and students with lower performance have more absences.

7 Machine Learning Operations

In this section, we will utilize machine learning techniques to predict the GradeClass column using all available features, excluding the GPA and Student ID. The Student ID is not relevant to the predictive model, while GPA is inherently linked to the range of each GradeClass. We will employ two models: Random Forest and Decision Tree.

Random Forest is chosen for several advantages:

1. It effectively handles large datasets with numerous variables, often running into the thousands [3].
2. It provides estimates for variable importance, helping us understand which features contribute most to predictions [3].
3. It can automatically balance datasets when one class is less frequent than others, improving model performance [3].
4. It is suitable for complex tasks, making it versatile for various applications [3].

On the other hand, Decision Tree is selected for its unique strengths:

1. It can handle mixed data types, allowing for flexibility in feature selection [4].
2. It is scalable, making it suitable for large datasets[4].
3. It demonstrates robustness against outliers, ensuring that extreme values do not unduly influence the model[4].

To evaluate these models, we will use four key metrics:

1. Accuracy, which answers the question: "How often is the model correct?" [5].
2. Precision, which answers the question: "How often are the positive predictions correct?" [5].
3. Recall, which answers the question: "Can the model find all instances of the positive class?" [5].
4. F1-Score, which is the harmonic mean of Precision and Recall.

7.1 Reading and Preparing the Data

First, we start by reading the dataset and preparing it for the models. This involves changing the data types to numeric and saving the results as new columns, as shown in Figure 23. We also import the necessary libraries to facilitate the model development process.

```

scala> val dataset = spark.read.option("header", "true").csv("C:\\spark\\Student_performance_data_.csv")
dataset: org.apache.spark.sql.DataFrame = [StudentID: string, Age: string ... 13 more fields]

scala> val finalDataset = dataset.select(
  | col("StudentID"),
  | col("Age").cast("integer").alias("Cleaned_Age"),
  | col("Gender"),
  | col("Ethnicity"),
  | col("ParentalEducation"),
  | col("StudyTimeWeekly").cast("double").alias("Cleaned_StudyTimeWeekly"),
  | col("Absences").cast("integer").alias("Cleaned_Absences"),
  | col("Tutoring"),
  | col("ParentalSupport"),
  | col("Extracurricular"),
  | col("Sports"),
  | col("Music"),
  | col("Volunteering"),
  | col("GradeClass").cast("integer").alias("Cleaned_GradeClass")
  | )
finalDataset: org.apache.spark.sql.DataFrame = [StudentID: string, Cleaned_Age: int ... 12 more fields]

scala> import org.apache.spark.ml.classification.{RandomForestClassifier, DecisionTreeClassifier}
import org.apache.spark.ml.classification.{RandomForestClassifier, DecisionTreeClassifier}

scala> import org.apache.spark.ml.feature.{VectorAssembler, StringIndexer, OneHotEncoder}
import org.apache.spark.ml.feature.{VectorAssembler, StringIndexer, OneHotEncoder}

scala> import org.apache.spark.ml.evaluation.MulticlassClassificationEvaluator
import org.apache.spark.ml.evaluation.MulticlassClassificationEvaluator

scala> import org.apache.spark.sql.SparkSession
import org.apache.spark.sql.SparkSession

scala> import org.apache.spark.sql.functions._
import org.apache.spark.sql.functions._

```

Figure 23 Reading the Data and Importing the libraries

After that, we use `StringIndexer`, which converts categorical string labels into numerical indices as shown in Figure 24. We specify the output column as `label`, corresponding to the `Cleaned_GradeClass`. Next, we fit the indexer using the `fit` method, which produces a new dataset named `indexedDataset`. This dataset includes the label column along with the original dataset.

```

scala> val indexer = new StringIndexer().setInputCol("Cleaned_GradeClass").setOutputCol("label")
indexer: org.apache.spark.ml.feature.StringIndexer = stridx_f2d049ca3db4

scala> val indexedDataset = indexer.fit(finalDataset).transform(finalDataset)
indexedDataset: org.apache.spark.sql.DataFrame = [StudentID: string, Cleaned_Age: int ... 13 more fields]

```

Figure 24 Data preparation using `StringIndexer` and `VectorAssembler`

Next, we define the categorical columns and map them using `StringIndexer`, appending the `"_index"` suffix to indicate the new indexed columns as shown in Figure 25. We create `tempDataset` to preserve the original data and serve as an intermediate step for processing. Next, we iterate over each indexer in the `indexers` array. For each indexer, we use the `fit` method on `tempDataset` to learn the mapping of the categorical values to numerical indices, and then we transform `tempDataset` to include the new indexed column. After that, we use the `OneHotEncoder` method for the categorical columns, appending the `"_ohe"` suffix to indicate the new one-hot encoded columns. This step transforms the indices into a binary vector representation, allowing the model to interpret categorical data effectively. We iterate over each encoder in the `encoders` array. For each encoder, we call the `fit` method on `tempDataset` to learn the mapping of the indexed values to a one-hot encoded representation. We then use the `transform` method to apply this mapping, resulting in a new `DataFrame` that includes the one-hot encoded columns. Finally, we update `tempDataset` to include these new one-hot encoded columns.

```
scala> val categoricalColumns = Array(
  |   "Gender",
  |   "Ethnicity",
  |   "ParentalEducation",
  |   "Tutoring",
  |   "ParentalSupport",
  |   "Extracurricular",
  |   "Sports",
  |   "Music",
  |   "Volunteering"
  | )
categoricalColumns: Array[String] = Array(Gender, Ethnicity, ParentalEducation, Tutoring, ParentalSupport, Extracurricular, Sports, Music, Volunteering)
scala> var tempDataset = indexedDataset
tempDataset: org.apache.spark.sql.DataFrame = [StudentID: string, Cleaned_Age: int ... 13 more fields]
scala> val indexers = categoricalColumns.map { column => new StringIndexer().setInputCol(column).setOutputCol(column + "_index")}
indexers: Array[org.apache.spark.ml.feature.StringIndexer] = Array(strIdx_21da2fc8593a, strIdx_68a792214e4e, strIdx_b10e2c44cb31, strIdx_3cf7795ec47a, strIdx_dc35f260f14a, strIdx_8c8a56baa209, strIdx_4f1d2f004eef, strIdx_d593a27470f5, strIdx_14275e5aaa8e)
scala> for (indexer <- indexers) {tempDataset = indexer.fit(tempDataset).transform(tempDataset)}
scala> val encoders = categoricalColumns.map { column => new OneHotEncoder().setInputCol(column + "_index").setOutputCol(column + "_ohe")}
encoders: Array[org.apache.spark.ml.feature.OneHotEncoder] = Array(oneHotEncoder_70fcfe9f9a81, oneHotEncoder_eba727be4a1e, oneHotEncoder_def35d2f197e, oneHotEncoder_6407b6529184, oneHotEncoder_629c1888e7d5, oneHotEncoder_8bf26bc18fb2, oneHotEncoder_106f8cf8e584, oneHotEncoder_cb9f52268b9e, oneHotEncoder_953804c31850)
scala> for (encoder <- encoders) {tempDataset = encoder.fit(tempDataset).transform(tempDataset)}
```

Figure 25 Categorical column indexing.

Then we define the feature columns variable as an array that includes the names of the one-hot encoded columns and any other relevant features as shown in Figure 26. Next, we use the `VectorAssembler` method to combine these feature columns into a single feature vector. The `VectorAssembler` takes the specified columns and creates a new column that contains a dense vector representation of the features, which is essential for feeding into machine learning models. We then call `transform` on `tempDataset` to apply the `VectorAssembler`, updating the dataset to include this new feature vector column. Then we prepare the training and test data by splitting the dataset, allocating 80% of the data for training and 20% for testing. This ensures that the model is trained on a substantial portion of the data while retaining a separate subset for evaluating its performance.

```
scala> val featureColumns = Array(
  |   "Cleaned_Age",
  |   "Gender_ohe",
  |   "Ethnicity_ohe",
  |   "ParentalEducation_ohe",
  |   "Cleaned_StudyTimeWeekly",
  |   "Cleaned_Absences",
  |   "Tutoring_ohe",
  |   "ParentalSupport_ohe",
  |   "Extracurricular_ohe",
  |   "Sports_ohe",
  |   "Music_ohe",
  |   "Volunteering_ohe"
  | )
featureColumns: Array[String] = Array(Cleaned_Age, Gender_ohe, Ethnicity_ohe, ParentalEducation_ohe, Cleaned_StudyTimeWeekly, Cleaned_Absences, Tutoring_ohe, ParentalSupport_ohe, Extracurricular_ohe, Sports_ohe, Music_ohe, Volunteering_ohe)
scala> val assembler = new VectorAssembler().setInputCols(featureColumns).setOutputCol("features")
assembler: org.apache.spark.ml.feature.VectorAssembler = VectorAssembler: uid=vecAssembler_91801f01e36b, handleInvalid=error, numInputCols=12
scala> val assembledDataset = assembler.transform(tempDataset)
assembledDataset: org.apache.spark.sql.DataFrame = [StudentID: string, Cleaned_Age: int ... 32 more fields]
scala> val Array(trainingData, testData) = assembledDataset.randomSplit(Array(0.8, 0.2), seed = 1234)
trainingData: org.apache.spark.sql.Dataset[org.apache.spark.sql.Row] = [StudentID: string, Cleaned_Age: int ... 32 more fields]
testData: org.apache.spark.sql.Dataset[org.apache.spark.sql.Row] = [StudentID: string, Cleaned_Age: int ... 32 more fields]
```

Figure 26 Defining feature columns and preparing the dataset for model training.

7.2 Random Forest

In this section, we will use Random Forest as the first model, as shown in Figure 27. We start by initializing the `RandomForestClassifier`. After that, we use the `fit` method to train the model on the training data. Next, we use the trained model to make predictions on the test dataset. We evaluate the model's performance using various metrics, including accuracy, precision, recall, and F1 score, which provide insights into the model's effectiveness in classifying the data. Finally, we display the predictions alongside the actual labels and the corresponding feature vectors for further analysis.

```
scala> val rf = new RandomForestClassifier().setLabelCol("label").setFeaturesCol("features").setNumTrees(10)
rf: org.apache.spark.ml.classification.RandomForestClassifier = rfc_67e5720403f3

scala> val rfModel = rf.fit(trainingData)
rfModel: org.apache.spark.ml.classification.RandomForestClassificationModel = RandomForestClassificationModel: uid=rfc_67e5720403f3, numTrees=10, numClasses=5, numFeatures=20

scala> val rfPredictions = rfModel.transform(testData)
rfPredictions: org.apache.spark.sql.DataFrame = [StudentID: string, Cleaned_Age: int ... 35 more fields]

scala> val evaluator = new MulticlassClassificationEvaluator().setLabelCol("label").setPredictionCol("prediction")
evaluator: org.apache.spark.ml.evaluation.MulticlassClassificationEvaluator = MulticlassClassificationEvaluator: uid=mcEval_907d675aa68d, metricName=f1, metricLabel=0.0, beta=1.0, eps=1.0E-15

scala> println(s"Random Forest Accuracy: ${evaluator.setMetricName("accuracy").evaluate(rfPredictions)}")
Random Forest Accuracy: 0.7071583514099783

scala> println(s"Random Forest Precision: ${evaluator.setMetricName("weightedPrecision").evaluate(rfPredictions)}")
Random Forest Precision: 0.6603687260790639

scala> println(s"Random Forest Recall: ${evaluator.setMetricName("weightedRecall").evaluate(rfPredictions)}")
Random Forest Recall: 0.7071583514099782

scala> println(s"Random Forest F1 Score: ${evaluator.setMetricName("f1").evaluate(rfPredictions)}")
Random Forest F1 Score: 0.6756108427643404

scala> rfPredictions.select("label", "prediction", "features").show(truncate = false)
+-----+-----+-----+
|label|prediction|features|
+-----+-----+-----+
|3.0|3.0|[20, [0, 2, 6, 9, 11, 14, 16, 17, 18, 19], [18.0, 1.0, 1.0, 1.0, 15.40875605584674, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]]|
|0.0|0.0|[20, [0, 1, 2, 5, 9, 10, 13, 16, 17, 18, 19], [17.0, 1.0, 1.0, 1.0, 4.6724952729713305, 17.0, 1.0, 1.0, 1.0, 1.0, 1.0]]|
|0.0|0.0|[20, [0, 2, 5, 9, 10, 11, 14, 17, 18, 19], [15.0, 1.0, 1.0, 1.0, 15.323142031655559, 25.0, 1.0, 1.0, 1.0, 1.0, 1.0]]|
|0.0|0.0|[20, [0, 1, 2, 6, 9, 10, 11, 12, 16, 17, 18, 19], [18.0, 1.0, 1.0, 1.0, 7.380354648223455, 15.0, 1.0, 1.0, 1.0, 1.0, 1.0]]|
|2.0|1.0|[20, [0, 1, 2, 7, 9, 10, 11, 15, 16, 17, 19], [16.0, 1.0, 1.0, 1.0, 2.71833747121508807, 5.0, 1.0, 1.0, 1.0, 1.0, 1.0]]|
|2.0|2.0|[20, [0, 2, 6, 9, 10, 11, 12, 16, 18, 19], [16.0, 1.0, 1.0, 1.0, 367992532661305, 2.0, 1.0, 1.0, 1.0, 1.0, 1.0]]|
|1.0|1.0|[20, [0, 1, 2, 7, 9, 10, 11, 12, 16, 17, 19], [16.0, 1.0, 1.0, 1.0, 2.2521845369804137, 8.0, 1.0, 1.0, 1.0, 1.0, 1.0]]|
|0.0|0.0|[20, [0, 2, 6, 9, 10, 14, 16, 17, 18, 19], [18.0, 1.0, 1.0, 1.0, 5.3041970909721402, 26.0, 1.0, 1.0, 1.0, 1.0, 1.0]]|
|2.0|1.0|[20, [0, 2, 5, 9, 10, 11, 15, 16, 17, 19], [18.0, 1.0, 1.0, 1.0, 4.8943119408474019, 7.0, 1.0, 1.0, 1.0, 1.0, 1.0]]|
|0.0|0.0|[20, [0, 2, 5, 9, 10, 11, 12, 16, 17, 18, 19], [17.0, 1.0, 1.0, 1.0, 6.04553406092897, 15.0, 1.0, 1.0, 1.0, 1.0, 1.0]]|
|0.0|0.0|[20, [0, 1, 2, 6, 9, 10, 11, 15, 16, 17, 19], [17.0, 1.0, 1.0, 1.0, 18.948559961195862, 22.0, 1.0, 1.0, 1.0, 1.0, 1.0]]|
|0.0|0.0|[20, [0, 1, 2, 9, 10, 11, 12, 16, 19], [18.0, 1.0, 1.0, 1.0, 9.213833935093518, 20.0, 1.0, 1.0, 1.0, 1.0, 1.0]]|
|0.0|0.0|[20, [0, 2, 9, 10, 11, 13, 17, 18, 19], [15.0, 1.0, 1.0, 1.0, 8.7755842322235, 29.0, 1.0, 1.0, 1.0, 1.0, 1.0]]|
|2.0|1.0|[20, [0, 2, 6, 9, 10, 13, 16, 18, 19], [17.0, 1.0, 1.0, 1.0, 6.771800548741026, 9.0, 1.0, 1.0, 1.0, 1.0, 1.0]]|
|1.0|1.0|[20, [0, 2, 6, 9, 10, 11, 12, 16, 18, 19], [15.0, 1.0, 1.0, 1.0, 6.680977369563614, 8.0, 1.0, 1.0, 1.0, 1.0, 1.0]]|
|0.0|0.0|[20, [0, 2, 9, 10, 11, 13, 16, 18, 19], [18.0, 1.0, 2.3951993286439444, 23.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]]|
|0.0|0.0|[20, [0, 1, 3, 6, 9, 10, 11, 13, 16, 17], [16.0, 1.0, 1.0, 1.0, 17.255415549184512, 28.0, 1.0, 1.0, 1.0, 1.0, 1.0]]|
|2.0|1.0|[20, [0, 3, 6, 9, 10, 11, 12, 18, 19], [15.0, 1.0, 1.0, 1.0, 3.258053089547881, 8.0, 1.0, 1.0, 1.0, 1.0, 1.0]]|
|2.0|2.0|[20, [0, 2, 5, 9, 10, 11, 13, 16, 17, 18, 19], [18.0, 1.0, 1.0, 1.0, 5.305701781721048, 3.0, 1.0, 1.0, 1.0, 1.0, 1.0]]|
|1.0|0.0|[20, [0, 1, 2, 5, 9, 10, 11, 13, 18, 19], [17.0, 1.0, 1.0, 1.0, 5.125847587384352, 12.0, 1.0, 1.0, 1.0, 1.0, 1.0]]|
+-----+-----+-----+
only showing top 20 rows
```

Figure 27 Random Forest Model

7.3 Decision Tree

In this section, we will use Decision Tree as the second model, as shown in Figure 28. We start by initializing the `DecisionTreeClassifier`. After that, we use `fit` method to fit the model to the training data. Next, we use the trained model to make predictions on the test dataset. We evaluate the model's performance using various metrics, including accuracy, precision, recall, and F1 score, which provide insights into the model's effectiveness in classifying the data. Finally, we display the predictions alongside the actual labels and the corresponding feature vectors for further analysis.

```
scala> val dt = new DecisionTreeClassifier().setLabelCol("label").setFeaturesCol("features")
dt: org.apache.spark.ml.classification.DecisionTreeClassifier = dtc_9b23f22082d9

scala> val dtModel = dt.fit(trainingData)
dtModel: org.apache.spark.ml.classification.DecisionTreeClassificationModel = DecisionTreeClassificationModel: uid=dtc_9b23f22082d9, depth=5, numNodes=47, numClasses=5, numFeatures=20

scala> val dtPredictions = dtModel.transform(testData)
dtPredictions: org.apache.spark.sql.DataFrame = [StudentID: string, Cleaned_Age: int ... 35 more fields]

scala> println(s"Decision Tree Accuracy: ${evaluator.setMetricName("accuracy").evaluate(dtPredictions)}")
Decision Tree Accuracy: 0.7049891540130152

scala> println(s"Decision Tree Precision: ${evaluator.setMetricName("weightedPrecision").evaluate(dtPredictions)}")
Decision Tree Precision: 0.7139686851218585

scala> println(s"Decision Tree Recall: ${evaluator.setMetricName("weightedRecall").evaluate(dtPredictions)}")
Decision Tree Recall: 0.7049891540130152

scala> println(s"Decision Tree F1 Score: ${evaluator.setMetricName("f1").evaluate(dtPredictions)}")
Decision Tree F1 Score: 0.6981649188845374

scala> dtPredictions.select("label", "prediction", "features").show(truncate = false)
+-----+-----+-----+
|label|prediction|features|
+-----+-----+-----+
|3.0|2.0|[20,0,2,6,9,11,14,16,17,18,19],[18.0,1.0,1.0,15.408756855884674,1.0,1.0,1.0,1.0,1.0,1.0]|
|0.0|0.0|[20,0,1,2,5,9,10,13,16,17,18,19],[17.0,1.0,1.0,1.0,4.67249527279713305,17.0,1.0,1.0,1.0,1.0,1.0]|
|0.0|0.0|[20,0,2,5,9,10,11,14,17,18,19],[15.0,1.0,1.0,15.323142031655559,25.0,1.0,1.0,1.0,1.0,1.0]|
|0.0|0.0|[20,0,1,2,6,9,10,11,12,16,17,18,19],[18.0,1.0,1.0,1.0,7.388354648223455,15.0,1.0,1.0,1.0,1.0,1.0]|
|2.0|1.0|[20,0,1,2,7,9,10,11,15,16,17,19],[16.0,1.0,1.0,1.0,2.7103374712150807,5.0,1.0,1.0,1.0,1.0,1.0]|
|2.0|2.0|[20,0,2,6,9,10,11,12,16,18,19],[16.0,1.0,1.0,1.0,10.367992332661303,2.0,1.0,1.0,1.0,1.0,1.0]|
|1.0|1.0|[20,0,1,2,7,9,10,11,13,16,17,19],[16.0,1.0,1.0,1.0,2.2521845869844137,8.0,1.0,1.0,1.0,1.0,1.0]|
|0.0|0.0|[20,0,2,6,9,10,14,16,17,18,19],[18.0,1.0,1.0,5.344197989721482,26.0,1.0,1.0,1.0,1.0,1.0]|
|2.0|1.0|[20,0,2,5,9,10,11,15,16,17,19],[18.0,1.0,1.0,4.894311940474019,7.0,1.0,1.0,1.0,1.0,1.0]|
|0.0|0.0|[20,0,2,5,9,10,11,12,16,17,18,19],[17.0,1.0,1.0,6.045530408992897,15.0,1.0,1.0,1.0,1.0,1.0]|
|0.0|0.0|[20,0,1,2,6,9,10,11,15,16,17,19],[17.0,1.0,1.0,1.0,18.948559961195862,22.0,1.0,1.0,1.0,1.0,1.0]|
|0.0|0.0|[20,0,1,2,9,10,11,12,16,19],[18.0,1.0,1.0,0.9213833935893518,20.0,1.0,1.0,1.0,1.0,1.0]|
|0.0|0.0|[20,0,2,9,10,11,13,17,18,19],[15.0,1.0,1.0,8.87755842322235,29.0,1.0,1.0,1.0,1.0,1.0]|
|2.0|1.0|[20,0,2,6,9,10,13,16,18,19],[17.0,1.0,1.0,6.771888548741026,9.0,1.0,1.0,1.0,1.0,1.0]|
|1.0|1.0|[20,0,2,6,9,10,11,12,16,18,19],[15.0,1.0,1.0,8.680977369563614,8.0,1.0,1.0,1.0,1.0,1.0]|
|0.0|0.0|[20,0,2,9,10,11,13,16,18,19],[18.0,1.0,2.305199226439444,22.0,1.0,1.0,1.0,1.0,1.0,1.0]|
|0.0|0.0|[20,0,1,3,6,9,10,11,13,16,17],[16.0,1.0,1.0,1.0,17.255415540184512,28.0,1.0,1.0,1.0,1.0,1.0]|
|2.0|2.0|[20,0,3,6,9,10,11,12,18,19],[15.0,1.0,1.0,3.258053089547881,8.0,1.0,1.0,1.0,1.0,1.0]|
|2.0|3.0|[20,0,2,5,9,10,11,13,16,17,18,19],[18.0,1.0,1.0,5.385701781721048,3.0,1.0,1.0,1.0,1.0,1.0]|
|1.0|0.0|[20,0,1,2,5,9,10,11,13,18,19],[17.0,1.0,1.0,1.0,5.125847507384352,12.0,1.0,1.0,1.0,1.0,1.0]|
+-----+-----+-----+
only showing top 20 rows
```

Figure 28 Decision Tree Model

7.4 Results

Table 2 presents a comparison between the two machine learning models which implemented which are Random Forest and Decision Tree. The two models had very similar accuracy, but the Decision Tree model had a slightly higher F1-score (0.698) compared to Random Forest (0.675) which indicates that it balances precision and recall. Both models had close recall, but Decision Tree was better than the Random Forest in precision. To conclude, both models were close and effective in predictions, but the Decision Tree model had a slight advantage in terms of F1-score and precision, so it is the better option for the task.

Table 2 Models Comparison

	Accuracy	F1-score	Recall	Precision
Random Forest	0.707	0.675	0.707	0.660
Decision Tree	0.704	0.698	0.704	0.713

8 Conclusion

In this project, we explored the Students Performance Dataset from Kaggle to understand the factors that influence student success. We started with essential preprocessing steps to ensure the data was clean and ready for analysis. Following that, we performed various RDD transformations and actions to reveal relationships within the dataset.

We then implemented two machine learning models: Random Forest and Decision Tree, to predict students' GradeClass. After evaluating these models using key metrics like accuracy, precision, recall, and F1-score, we found that the Decision Tree model had a slight edge in balancing precision and recall.

These results emphasize the importance of understanding what affects academic performance. They suggest that tailored interventions could improve student outcomes. Looking ahead, future efforts could focus on leveraging these insights to create effective educational strategies that enhance student success.

9 References

- [1] R. El Kharoua, "Students Performance Dataset," Kaggle, 2021. [Online]. Available: <https://www.kaggle.com/datasets/rabieelkharoua/students-performance-dataset>.
- [2] Deepa Vasanthkumar, "Spark dataframes select vs withcolumn comparison," Medium, Jun. 10, 2024. [Online]. Available: <https://medium.com/@deepa.account/spark-dataframes-select-vs-withcolumn-comparison-59512582ed5f>.
- [3] Corporate Finance Institute, "Random Forest - Definition, Algorithms With Examples," 2023. [Online]. Available: https://corporatefinanceinstitute.com/resources/data-science/random-forest/?utm_source=x&utm_medium=cpc&utm_campaign=PMax_SA&utm_term=&utm_content=&utm_matchtype=&utm_device=c&utm_ad=&campaign=PMax_SA&adgroupid=&keyword=&device=c&network=x&placement=&adp.
- [4] E. Newman, "8 Key Advantages of Decision Tree," Yonyx, Jun. 13, 2024. [Online]. Available: <https://corp.yonyx.com/automate-support/advantage-of-decision-tree/>.
- [5] Evidently AI, "Accuracy vs. precision vs. recall in machine learning: what's the difference?," Jan. 9, 2025. [Online]. Available: <https://www.evidentlyai.com/classification-metrics/accuracy-precision-recall#:~:text=You%20can%20calculate%20accuracy%20by,higher%20the%20accuracy%2C%20the%20better>.